Appello 18 Luglio 2016



Politecnico di Milano

Anno accademico 2015-2016

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Ghezzi	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta il divieto di partecipazione all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:
Esercizio 5:

Esercizio 1

La classe mutabile `Sequence` rappresenta una lista sequenziale di elementi, di un tipo Dato, descritto parzialmente piu' avanti. La sequenza non e' mai vuota, ossia contiene sempre almeno un elemento. Una specifica informale di `Sequence` e' data di seguito, con il metodo affinita' che e' spiegato piu' avanti.

```
public class Sequence {

 //costruisce una sequenza di un solo dato.
 public Sequence(Dato d)

 //restituisce il numero di elementi nella sequenza
 public int size()

 //suppone che  d non \'e nullo
 //inserisce un nuovo dato in ultima posizione
 public void insert(Dato d)

 // restituisce il dato in posizione i, se 0<=i<size()
 //altrimenti lancia IndexOutOfBoundException
 public Dato get(int i) throws IndexOutOfBoundException

 //calcola l'affinita' totale della sequenza, ignorando i dati duplicati
 public double affinitaTotale()

 //cancella dalla sequenza l'occorrenza piu' a sx di d,
 //ma lancia eccezione EmptyException se d e' l'unico elemento.
 public void delete(Dato d) throws EmptyException

...
}
```

Ogni elemento di tipo Dato ha un'affinita' rispetto ad ogni altro elemento di tipo Dato. L'affinita' e' un valore reale compreso fra 0 e 1. L'affinita' massima (uno) e' raggiunta solo con gli elementi per cui vale `equals`, ossia `d1.affinita(d2) = 1` se e solo se `d1.equals(d2)`. La classe immutabile `Dato` ha, oltre a un costruttore non meglio specificato, un metodo `double affinita(Dato d)`, che calcola l'affinita di this con il dato d. L'affinita' totale di una sequenza e' data dal prodotto delle affinita' delle coppie di dati contigue. Es. se la sequenza e' `d1 d2 d3`, allora `affinitaTotale` vale `affinita(d1,d2)*affinita(d2,d3)`.

Parte (a)

1. Si scriva la specifica JML dei metodi `affinitaTotale()` e `delete`.

   SOLUZIONE:

```
//@requires size()>1
//@ensures \result ==
//@    (\product int i; 0<=i<size()-1;
//@          (\exists int j; 0<=i && i<j;
//@               get(i).equals(get(j)) && get(i+1).equals(get(j+1)))
//@          ? 1
//@          : get(i).affinita(get(i+1)))
public /*@ pure @*/ double affinitaTotale()
```

   Def. per semplicita' un metodo minPos che restituisce la posizione piu' a sx di un dato d, -1 se non esiste:

```
//@requires d!=null;
//@ensures \result ==
//@       (\exists int i;  0<=i && i<size(); get(i).equals(d))
//@       ? (\min int i; 0<=i<size()&& get(i).equals(d); i)
//@       : -1
public /*@ pure @*/ boolean minPos(Dato d)

//@ensures \old(size()) > 1 &&
//@    minPos(d)>-1
//@    ?   size()==\old(size()-1) &&
//@        (\forall int j; 0<=j && j<size();
//@             get(j).equals(\old(get(j<minPos(d) ? j : j+1))))
//@    : size()==\old(size() &&
//@      (\forall int j; 0<=j<size(); \old(get(j)).equals(get(j))))
//@signals(EmptyException e) \old(size())==1 && get(0).equals(\old(get(0))
public void delete(Dato d)
```

2. Si definisca in JML l'invariante pubblico della classe `Sequence`, in base alle specifiche e alle descrizioni fornite

   SOLUZIONE:

```
//@ public invariant size()>0 &&
//@        (\forall int i; 0<=i<size(); get(i)!=null) &&
//@        affinitaTotale()>=0 &&
//@        affinitaTotale()<=1
```

3. Si stabilisca, argomentando in modo rigoroso, se l'invariante pubblico è verificato dalla specifica. **SOLUZIONE:**

   La condizione nel public invariant che affinitaTotale() e' fra 0 e 1 deriva dalla specifica di affinitaTotale(), per cui il valore restituito e' sempre un prodotto di affinita', che per ipotesi sono comprese fra 0 e 1. La condizione $size() > 0$ e' verificata in quanto il costruttore costruisce una sequenza non vuota e l'unico metodo che puo' ridurre la dimensione della sequenza (ossia delete) lancia un'eccezione se $size() == 1$, senza cancellare l'elemento: quindi se anche la nuova size() diminuisce di 1, sara' sempre positiva.

Parte (b)

Si consideri un'implementazione di `Sequence` in cui si utilizza una LinkedList per memorizzare gli elementi in sequenza. La seguente classe Elemento, nello stesso package di Sequenza, contiene un dato e un riferimento all'elemento successivo, in modo da definire un elemento della LinkedList. Se un Elemento e' l'ultimo, allora il campo `next==null`.

```
class Elemento{
     Dato info;
     Elemento next;
     Elemento(Elemento n, Dato x) {
          next= n;
          info =x;
     }

public class Sequence{
   //rep
   private Elemento inizio;
   private Elemento fine;
```

```
   private double affinitaTot;

   public Sequenza(Dato d) {
    inizio= new Elemento(null,d);
    fine = inizio;
    affinitaTot=1;
  }
   public void insert(Dato d) {
     affinitaTot= affinitaTot* fine.info.affinita(d);
     fine.next = new Elemento(null, d);
     fine = fine.next;
     }
...
  }
}
```

1. (FACOLTATIVO) Scrivere l'invariante rappresentazione della classe `Sequence`, in base al frammento di implementazione dato.

   SOLUZIONE:
   Def. un metodo puro ausiliario getElem:

   ```
   //@requires 0<= i && i>0 ==> e.next!=null
   //@ensures (i==0|| e==null) ? \result == e :
   //@ \result ==getElem(Elem.next, i-1);
   public static /*@ pure @*/ Elemento getElem(Elemento e, int i)
   ```

   I'inv. asserisce che la lista e' sequenziale:

   ```
   private invariant inizio!=null && fine != null && fine.next == null &&
                     affinitaTot>=0 && affinitaTot<=1 &&
      (\forall int i; 0<=i && getElem(inizio,i)!=null;
          (\forall int j; i<j && getElem(inizio,j)!=null;
              getElem(inizio,i)!= getElem(inizio,j) )) &&
      (\exists int j; 0<=j; fine=getElem(inizio,j));
   ```

2. Definire la funzione di astrazione della classe `Sequence`. Si puo' usare un private invariant o scrivere un'implementazione del metodo toString().

   SOLUZIONE:
   per scrivere un private invariant, si puo' riutilizzare il metodo puro ausiliario getElem:

   ```
   private invariant
      (\forall int i; 0<=i && getElem(inizio,i)!=null;
          getElem(inizio,i)== get(i));
   ```

   in alternativa, si puo' definire la toString:

   ```
   /@override
   public String toString() {
     Elemento e = inizio;
     String s = "Sequenza: e.info";
     while (e.next!= null) {
       e=e.next;
       s = s+ ", " + e.info;
   ```

```
  }
  return s;
}
```

Parte (c)

Si consideri una variante di `Sequence`, detta `ThinSequence`, che ha anche un metodo che consente di cancellare un dato da una posizione assegnata, cosi' definito:

```
//@ensures (*elimina il dato in posizione i *)
//@signals (IndexOutOfBoundException e) i<0 || i>=size();
public void deleteAt(int i) throws IndexOutOfBoundException
```

1. Completare la specifica JML del metodo deleteAt, formalizzandone la **postcondizione**. **SOLUZIONE:**

   ```
   //@requries size()>0;
   //@ensures (\forall int j; 0<=j && j<size();
   //@             get(j).equals(\old(get(j<i ? j : j+1)))) &&
   //@    size()==\old(size()-1);
   ```

2. E' possibile definire questa classe `ThinSequence` come derivata da `Sequence`, o viceversa, rispettando il principio di sostituzione? Motivare la risposta.

Non e' possibile che `ThinSequence` erediti da `Sequence`, in quanto viene violata la regola delle proprieta' (l'invariante di `Sequence` prevede che una `Sequence` non sia mai vuota). Anche il viceversa non e' possible, in quanto una classe `Sequence` definita come erede di `ThinSequence` non verificherebbe il proprio invariante.

Esercizio 2

Si consideri la seguente classe che calcola le radici di una equazione di secondo grado. Il metodo che calcola le radici contiene un errore nel ramo commentato. E' invece una scelta di progetto restuire sempre due radici, anche nel caso in cui la radice e' una sola (in tal caso i due valori restituiti sono uguali) o non esistono radici reali (in tal caso il valore restituito e' -1).

```
//calcolo radici di ax^2+bx+c come (-b+-sqrt(b^2-4ac))/(2a)
class Roots {
     double root1, root2;
     int num_roots;
     public  Roots(double a, b, c) {
          double q, r;
          q=b*b-4*a*c;
          if (q>0 && a!=0) {
               num_roots = 2;
               r = (double) Math.sqrt(q);
               root1= (r-b)/(2*a);
               root2=(-r-b)/(2*a);
          } else if (q==0) {//in questo ramo c'e' un errore: l'assunzione che
                    //esista esattamente 1 radice
               num_roots=1;
               root1=(-b)/(2*a);
               root2=root1;
          } else {
               num_roots=0;
               root1=-1;
               root2=-1;
          }
     }
     public  int num_roots() {return num_roots;}
     public  double first_root() {return root1; }
     public  double second_root() {return root2; }
}
```

1. Fornire un insieme minimo di casi di test che coprono tutte le istruzioni (statements);
2. L'insieme minimo di cui sopra copre anche tutti i branch?
3. L'insieme minimo di cui sopra soddisfa anche il criterio di condition coverage? In caso negativo, fornire un insieme minimo che soddisfi il criterio.
4. L'insieme di casi di test fornito in risposta ai punti 2 e 3 consente di rilevare l'errore presente nel codice? In caso negativo, quale test addizionale deve essere fornito per rilevare l'errore?
5. Si supponga di generare invece i casi di test in maniera casuale. Ipotizzando che il computer abbia parole di 64 bit per rappresentare il valori numerici double, qual'e' la probabilita' che si generino dati che portano ad attraversare il ramo che contiene l'errore e a generare un malfunzionamento?

SOLUZIONE:
Per il punto 5: deve essere $q = 0$ per attraversare il ramo errato e $a = 0$ per causare l'errore. Quindi solo $a = 0 \wedge b = 0$ portano alla generazione del malfunzionamento. Scegliendo i valori a e b in modo casuale, la prob sarebbe $\frac{1}{2^{64} \times 2^{64}} = 2^{-128}$.

Esercizio 3

Si consideri un'applicazione di commercio elettronico per la gestione degli ordini di acquisto. Un ordine, una volta creato dal cliente, puo' essere da pagare, pagato, cancellato, spedito. Gli ordini non pagati o cancellati non possono essere spediti, mentre quelli spediti non possono essere cancellati o spediti di nuovo.

a) Si definisca una struttura (ad es. in UML) per rappresentare una soluzione estendibile alla gestione degli ordini, utilizzando design pattern opportuni. Accennare anche all'implementazione in Java di classi significative.

b) Si delinei come estendere la soluzione precedente a nuovi requisiti. In particolare, la gestione deve considerare che gli ordini spediti possono andare in restituzione (ad esempio, perche' il materiale ordinato e' guasto o il cliente esercita il diritto di recesso).

SOLUZIONE:

parte (a): Si tratta di applicare il pattern State.

```
public interface StatoOrdine   {
        boolean pagato(Ordine ordine);
        StatoOrdine spedisci(Ordine ordine);
        boolean cancellabile(Ordine ordine);
        StatoOrdine cancella(Ordine ordine);
        SituazioneOrdine status();
}

public class Ordine   {
        public final static DA_PAGARE=0, SPEDITO=1, CANCELLATO=2, PAGATO=3;
        private StatoOrdine _statoOrdine;

        public Ordine(StatoOrdine statoOrdine) {
            _statoOrdine = statoOrdine;
        }

        int idOrdine;
        String cliente;
        Date dataOrdine;
        public int status()     {
            return _statoOrdine.status();
        }
        public boolean cancellabile()        {
            return _statoOrdine.cancellabile(this);
        }
        public void cancella()        {
            if (cancellabile())
                _statoOrdine.cancella(this);
        }
        public boolean pagato()        {
            return _statoOrdine.pagato(this);
        }
        public void spedisci()        {
            if (pagato())
                _statoOrdine.spedisci(this);
        }

        void modifica(StatoOrdine statoOrdine)        {
            _statoOrdine = statoOrdine;
        }
```

```
    }
```

Poi ad esempio uno stato puo' essere implementato in questo modo:

```
public class StatoCancellato implements StatoOrdine
    {
        public boolean pagato(Ordine ordine)    {
            return false;
        }

        public StatoOrdine spedisci(Ordine ordine)  {
            throw new UnaNuovaException("non Spedibile! Ordine cancellato!");
        }

        public boolean cancellabile(Ordine ordine)       {
            return false;
        }

        public StatoOrdine cancella(Ordine ordine) {
            throw new UnaNuovaException("Ordine gia' cancellato!");        }

        public int status()     {
                  return Ordine.CANCELLATO;
         }
    }
```

Parte (b). La via piu' semplice e'

- definire una nuova interfaccia StatoOrdineRestituibile, con il nuovo metodo restituisci(...), che eredita da StatoOrdine;
- implementare la nuova interfaccia con una classe StatoRestituito,
- definire una nuova classe OrdineRestituibile che eredita da Ordine e gestisce la restituzione ridefinendo opportunamente i metodi secondo le modalita' di gestione della restituzione.

Esercizio 4

Sono dati i seguenti frammenti di codice, in cui il significato delle variabili e' autoesplicativo:

```
long primesCount = 0;
for(long number : numbers) {
    if(isPrime(number))
        primesCount += 1;
}
```

```
int max = 0;
for(int price : prices) {
  max = Math.max(max,price);
}
```

Riscrivere i frammenti usando la notazione funzionale di Java 8.
SOLUZIONE:

```
final long primesCount =
           numbers .stream()
                   .filter(number -> isPrime(number)) .count();
```

```
final int max = prices.stream().reduce(0, Math::max);
```

Esercizio 5

Si consideri il seguente programma Java

```
1   class Hotel {
2      public int bookings;
3      public void book() {
4         bookings++;
5      }
6   }
7   public class SuperHotel extends Hotel {
8      public void book() {
9         bookings--;
10     }
11     public void book(int size) {
12        book();
13        super.book();
14        bookings += size;
15     }
16     public static void main(String args[]) {
17        Hotel hotel = new Hotel();
18        hotel.book(2);
19        System.out.print(hotel.bookings);
20     }
21   }
```

Domanda 1

Questo programma genera un qualche errore? Se sì, si tratta di uno (o più) errori segnalati a compile-time o a run-time? Se sono presenti errori, come correggerli? Scegliere la/le alternativa/e e motivarla/e opportunamente.

A. Aggiungere un parametro intero al metodo dichiarato alla linea 3.

B. Eliminare il parametro 2 alla linea 18.

C. Cambiare il tipo dinamico dell'oggetto hotel alla riga 17 in SuperHotel

D. Cambiare il tipo statico dell'oggetto hotel alla riga 17 in SuperHotel

E. Entrambe la C e D.

F. Nessuna correzione necessaria.

SOLUZIONE:

Il programma non compila a causa della chiamata alla linea 18 con un parametro int, non previsto nella segnatura di Hotel.book().

Esaminiamo alternative:

A) da sola causa errore di compilazione alla riga 13, dove ci si aspetta che super.book() non abbia parametri.

B) da sola e' sufficiente perche' il programma compili e si esegua correttamente.

C) da sola causa errore di compilazione

D) da sola causa errore di compilazione

E) C e D assieme sono sufficienti perche' il programma compili e si esegua correttamente.

F) e' ovviamente errata.

La risposta e' quindi che sono corrette le alternative B ed E.